GUÍA CONTROL CHECK 1

1. Entidades

1.1 Creación

Lo primero es añadir @Entity para convertir la clase en una entidad.

Para la Id de la entidad haremos que la clase herede BaseEntity, que ya trae el id Integer con @Id

Para cada restricción, disponemos de las siguientes anotaciones:

- @NotNull: obligatorio/no puede ser nulo.
- @Size(min=X, max=Y): longitud mínima X y máxima Y.
- @NotBlank: no puede estar vacío (texto).
- @Enumerated(EnumType.STRING): enumerado guardado como cadena.
- @Min(X) y @Max(Y): número entre X e Y.
- @Past: fecha en el pasado.

1.2 Relaciones entre las entidades

Lo primero es eliminar @Transient. Según el UML:

UML (lado origen → destino)	Anotación	Restricción extra según el destino
0..n → 1	@ManyToOne	1 → @NotNull
0..n → 0..1	@ManyToOne	
0..n → 0..n	@ManyToMany	
0..n → 2..n	@ManyToMany	@NotNull + @Size(min=2)
1 → 1	@OneToOne	@NotNull
1 → 0..n	@OneToMany	

Escaneado con CamScanner

2. Script data.sql

INSERT INTO nombre_tabla (atributos, de, la, tabla)
VALUES (valores, que, queramos, insertar).

Para la tabla de asociación generada con el @ManyToMany:
El nombre de la tabla será: nombre_tabla_original_nombre_fk

Ejemplo: ChallengeAnnouncement $\overset{0..n}{\longrightarrow}\overset{?..n}{}$ User

```
@ManyToMany
List<User> recipients;
```

↑ Crea la tabla: challenge_announcement_recipients

3. Servicio transaccional

Un servicio lleva @Service en la clase y @Transactional en cada método.

- Obtener todas las partidas usando el repositorio:

```
@Transactional(readOnly = true)
public List<ChessMatch> findAll() {
    return repo.findAll();
}
```

- Obtener una partida por Id:

```
@Transactional(readOnly = true)
public Optional<ChessMatch> findById(Integer id){
    return repo.findById(id);
}
```

- Grabar partida en base-de datos, comprobando que winner no es nulo y su valor coincide con los jugadores. Lanzar InvalidWinner Exception en caso contrario.

```
@Transactional(rollbackFor = InvalidWinnerException.class)
public ChessMatch save(ChessMatch match) throws InvalidWinnerException {
    User winner = match.getWinner();
    if(winner != null && !winner.equals(match.getWhite()) && !winner.equals(match.getBlack())
        throw new InvalidWinnerExreption();
    }
    return repo.save(match);
}
```

Escaneado con CamScanner

4. Endpoint REST con 200, 404 y 403

Las comprobaciones van en el servicio y el controlador delega. ResourceNotFoundException → 404
AccessDeniedException → 403

Servicio: realiza las comprobaciones necesarias

```
@Transactional(readOnly = true)
public List<RankingEntry> getRanking(Integer challengeId){
  //¿Existe?
  FriendlyChallenge challenge = findById(challengeId);
  //¿El usuario logado participa?
  User currentUser = userService.findCurrentUser();
  boolean isParticipant = challenge.getParticipants().stream()
      .anyMatch(p->p.getId().equals(currentUser.getId()));
  if(!isParticipant) throw new AccessDeniedException("Not a participant");
  //¿Hay partidas terminadas?
  List<Object[]> rows = repo.findParticipantPoints(challengeId);
  if(rows.isEmpty()) throw new ResourceNotFoundException("Not finished matches");
  return rows.stream()
      .map(row -> new RankingEntry((User)row[0], (Long)row[1]))
      .sorted(Comparator.comparingLong(RankingEntry::getPoints).reversed())
      .toList();
}
```

Controlador:

```
@GetMapping("/{id}/ranking")
public ResponseEntity<List<RankingEntry>> getRanking(@PathVariable("id") Integer id){
    return new ResponseEntity<>(servicio.getRanking(id), HttpStatus.OK);
}
```

Escaneado con CamScanner

5. React.

Pedimos los componentes con fetch dentro de useEffect

```
const [ranking, setRanking] = useState([]);

useEffect(() => {                                    //fetch al montar
    fetch(`/api/v1/challenges/${challengeId}/ranking`)
      .then((response) => response.json())
      .then((data) => setRanking(data));
}, [challengeId]);

const leader = ranking.length > 0 ? ranking[0] : null;

return(
  <div>
    <h1>Challenge info</h1>
    <h3>Ranking</h3>
    {leader &&
      <div>Leader: {leader.user.username} ({leader.points} points)</div>}
  </div>
);
}
```

Escaneado con CamScanner